Модуль 3. Улучшение качества кластеризации

3.1. Метод k-Nearest Neighbors

3.2. Визуализация полученных моделью результатов и Decision Tree

3.3. Визуализация результатов работы алгоритма градиентного спуска

Цель модуля

Научиться использовать визуализации для быстрого улучшения качества классификации.

Введение

Во время анализа данных и машинного обучения нередко возникают ситуации, в которых необходимо получить качественные результаты. Чаще всего это происходит при работе с алгоритмами кластеризации. Тогда используют продвинутые методы визуализации данных.

Задача этого модуля — научиться использовать визуализации для быстрого улучшения качества классификации.

Для чего это нужно:

- Это поможет сконцентрировать внимание на улучшении качества полученных результатов и не переживать о поиске наиболее эффективных инструментов визуализации.
- Наглядно следить за тем, как доработка модели влияет на качество результатов.

Мы обсудим применение визуализации при работе с алгоритмами k-Nearest Neighbors и Random Forest, а также при выводе результатов, полученных в модели. Все перечисленные модели вы уже изучали на курсе по машинному обучению, поэтому мы не будем фокусироваться на теории.

Наша цель — реализовать собственный алгоритм и разобраться, насколько возможно использовать визуализацию с собственными алгоритмами.

Представим, что мы работаем в большом агрохолдинге, который выращивает на продажу декоративные растения: тюльпаны, ирисы и розы. У нас есть проблема: мы закупаем сырье у местных фермеров, а они часто путают сорта цветов. В компании есть штат специалистов, которые вручную проверяют партии, но это долго и дорого.

Наша задача — собрать алгоритм, который при приемке будет делать это автоматически. На самом деле, это могут быть алгоритмы определения спелости помидоров или качества руды; просто у нас есть классический датасет с ирисами, собранный лично Рональдом Фишером, поэтому мы работаем с агрохолдингом.

3.1. Метод k-Nearest Neighbors

Начнем повторение алгоритмов с наиболее распространенного метода классификации в ML — **k-ближайших соседей (kNN)**. Его достаточно просто реализовать в отличие от других алгоритмов, поэтому для наглядности будем действовать следующим образом:

1. Напишем собственную реализацию и посмотрим на результаты, применив метод к стандартному датасету Iris.
2. Сравним полученные данные с библиотечной реализацией из sklearn.

Важно! Метод kNN относится к разряду ленивых классификаторов. Это значит, что метод обучается лишь на новых данных, не учитывая предыдущий опыт. То есть в процессе обучения алгоритм ничего не делает, лишь сохраняет размеченные тренировочные данные. Сам процесс классификации начинается только тогда, когда появляются новые немаркированные данные — алгоритм по определенным признакам считает расстояния между уже размеченными и еще неизвестными объектами в пространстве.

Импортируем датасет Iris из библиотеки sklearn. Он представляет собой собрание морфологических измерений разных сортов ирисов. Каждое растение имеет четыре характеристики:

- длина чашелистика;
- ширина чашелистика;
- длина лепестка;
- ширина лепестка.

Изначально этот датасет был собран Рональдом Фишером в 1936 году. Мы обязаны ему половиной статистического аппарата, который до сих пор используем. Он также придумал дисперсию и F-test.

```
from sklearn.datasets import load_iris
iris_dataset = load_iris() # Возвращает объект с несколькими полями
```

Посмотрим, что включает набор данных:

```
iris_dataset.keys()
```

```
dict_keys(['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module'])
```

Судя по тому, что работает метод .keys(), датасет представлен в формате аналога словаря из sklearn. Поэтому, к сожалению, нельзя использовать все те методы, которые доступны для pandas.DataFrame.

Посмотрим, в каком формате представлен словарь:

```
type(iris_dataset)
```

```
account_circle
```

```
sklearn.utils._bunch.Bunch
```

Мы имеем дело с отдельным типом данных Bunch, который реализован в библиотеке sklearn. Для удобства работы нужно посмотреть, к какому типу относятся отдельные переменные внутри этой структуры. Если они не подойдут нам, то нужно изменить форматирование.

Теперь посмотрим, какие типы ирисов представлены в датасете и какие фичи нам доступны:

```
iris_dataset['target_names'] # Виды ирисов
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

Мы видим ключевое слово array. Этот столбец представляет собой массив из библиотеки NumPy.

```
iris_dataset['feature_names'] # Характеристики каждого цветка
```

```
cle

['sepal length (cm)',

 'sepal width (cm)',

 'petal length (cm)',

 'petal width (cm)']
```

А вот этот столбец уже обычный Python-список. Теперь посмотрим разметку датасета и количество данных:

```
print(iris_dataset['target'].shape) # Каждый цветок соответствует виду

iris_dataset['target'] # 0 — setosa, 1 — versicolor, 2 — virginica
```

```
(150,)


array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```
print(type(iris_dataset['data']))

iris_dataset['data'].shape # Всего 150 цветков, у каждого 4 измерения
```

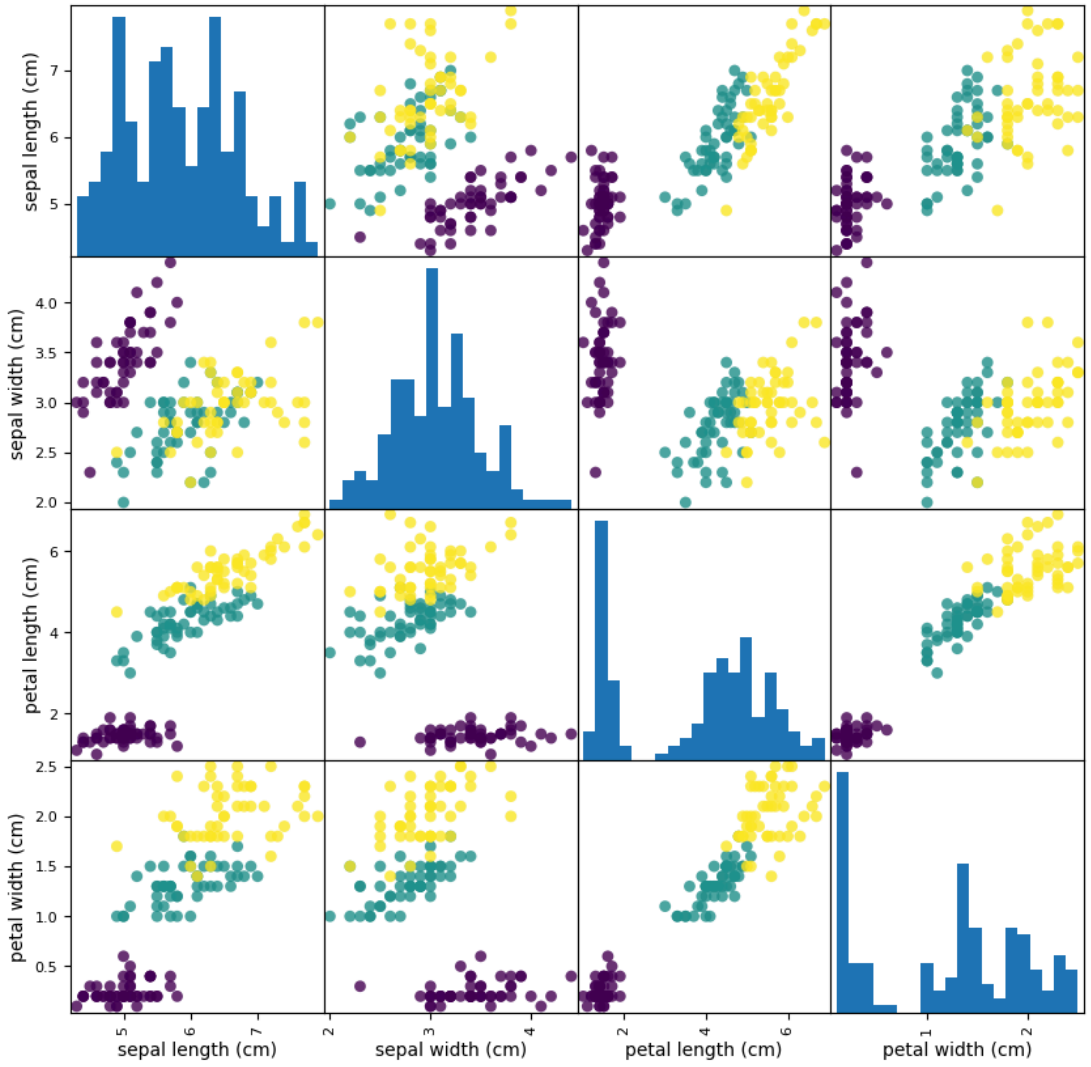
```
<class 'numpy.ndarray'>

(150, 4)
```

Перед анализом перейдем к главному — построим визуализацию, которая позволит зрительно оценить различия среди разных видов ириса. Ранее мы обсуждали модуль pandas.plot библиотеки Pandas. В этой библиотеке также есть более продвинутый метод — **pandas.plotting** — который позволяет строить графики на основе всего датасета, а не отдельных столбцов.

Один из таких графиков называется **scatter_matrix**. Он позволяет визуализировать корреляцию между всеми числовыми характеристиками в датасете. Как и многие датасеты, он поддерживает возможность окрашивания каждой точки в разный цвет в соответствии с данными столбцов. Эта характеристика может быть как категориальная, так и численная.

Поскольку для работы необходим DataFrame, сконвертируем датасет:

```
import pandas as pd


iris_dataframe = pd.DataFrame(iris_dataset['data'], columns=iris_dataset.feature_names)

scat_mtrx = pd.plotting.scatter_matrix(iris_dataframe, c=iris_dataset['target'], figsize=(10, 10), marker='o',

hist_kwds={'bins': 20}, s=40, alpha=.8)
```

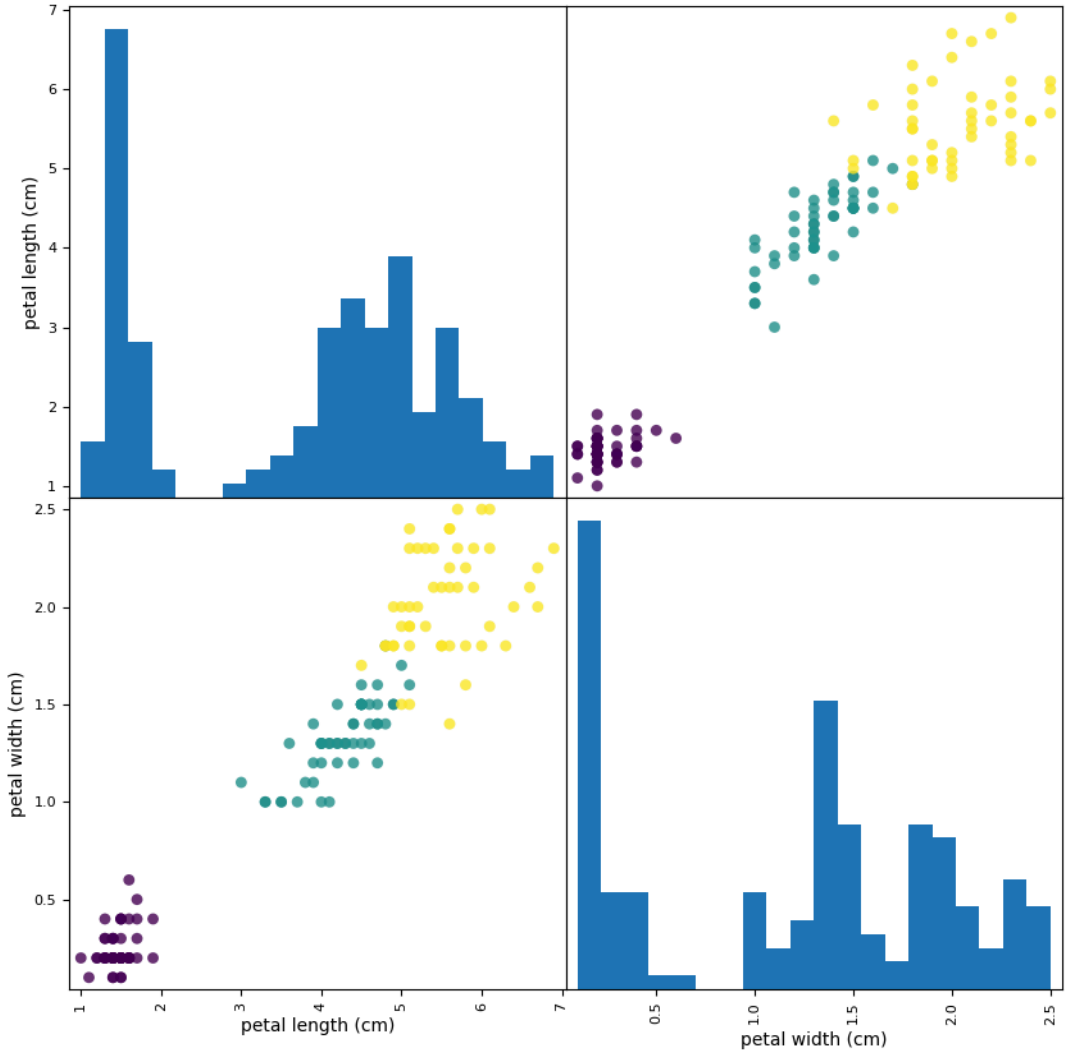


Что видно на графиках:

- По диагонали расположены не корреляции, а гистограммы распределения каждой переменной (может пригодиться при анализе).
- В графиках с точками показана зависимость разных показателей друг от друга.
- Разными цветами выделены разные сорта ириса. Уже визуально мы видим, что сорт, помеченный фиолетовым, сильно отличается от остальных. Это значит, что наш алгоритм должен хорошо сработать.
- Данные классов хорошо сепарируются по измерениям лепестков и чашелистиков. Скорее всего, модель машинного обучения сможет научиться неплохо их разделять. Этот вывод можно сделать еще до построения самой модели.

Но при четырех параметрах достаточно сложно представить, как расположены объекты относительно друг друга, так как придется работать в четырехмерном пространстве. По графикам видно, что цветки лучше всего разбиваются по измерениям длины и ширины лепестка (petal length, petal width), поэтому для наглядности оставим только эти данные.

```
iris_dataframe_simple = pd.DataFrame(iris_dataset.data[:,
2:4], columns=iris_dataset.feature_names[2:4])

scat_mtrx = pd.plotting.scatter_matrix(iris_dataframe_simple,
c=iris_dataset['target'], figsize=(10, 10), marker='o',

hist_kwds={'bins': 20}, s=40, alpha=.8)
```



Разобьем данные на тренировочный и тестовый датасеты. Для простоты реализации алгоритма объединим массивы признаков объектов и метки их классов. Так будет понятно, к какому классу относится каждый объект.

```
from sklearn.model_selection import train_test_split


x_train, x_test, y_train, y_test =
train_test_split(iris_dataset.data[:, 2:4],

iris_dataset['target'],

random_state=0) # random_state — для воспроизводимости


print(f'X_train shape: {x_train.shape}, y_train shape:
{y_train.shape},\n'

f'X_test shape: {x_test.shape}, y_test shape: {y_test.shape}')
```

```
X_train shape: (112, 2), y_train shape: (112,), X_test shape:
(38, 2), y_test shape: (38,)
```

```
import numpy as np

x_train_concat = np.concatenate((x_train, y_train.reshape(112,
1)), axis=1)

x_test_concat = np.concatenate((x_test, y_test.reshape(38,
1)), axis=1)

print(f'X_train shape: {x_train_concat.shape},\n'

f'X_test shape: {x_test_concat.shape}')
```

```
X_train shape: (112, 3), X_test shape: (38, 3)
```

```
x_train_concat_df = pd.DataFrame(x_train_concat)

x_test_concat_df = pd.DataFrame(x_test_concat)

x_train_concat_df.head(5)
```

0	1	2	
0	4.2	1.5	1.0
1	4.0	1.2	1.0
2	5.5	2.1	2.0
3	1.3	0.2	0.0
4	5.1	2.3	2.0

Теперь в последнем столбце у нас присутствуют метки класса.

Приступим к реализации алгоритма. Для начала обозначим метрику, по которой определим расстояние между объектами. Обозначим через $x = (x_1, x_2, \ldots, x_n)$ координаты объекта x в *n*-мерном пространстве, а через $y = (y_1, y_2, \ldots, y_n)$ — координаты объекта y.

По умолчанию алгоритм использует метрику Минковского. Если степень p = 2, она обращается в Евклидову метрику — расстояние между двумя точками в пространстве:

$$dist = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \ldots + (x_n - y_n)^2}$$

Ее и будем использовать:

```
import math


def euclidean_distance(data1, data2):
distance = 0
for i in range (len(data1) - 1):
distance += (data1[i] - data2[i]) ** 2
return math.sqrt(distance)
```

Вычислим расстояния до всех точек обучающей выборки и отберем k-соседей (то есть тех, расстояния до которых минимальны).

```
def get_neighbors(train, test, k=1):

distances = [(train[i][-1], euclidean_distance(train[i],
test))

for i in range (len(train))]

distances.sort(key=lambda elem: elem[1])


neighbors = [distances[i][0] for i in range (k)]

return neighbors
```

Теперь получим прогноз на основе классов соседей. Посчитаем, сколько объектов каждого класса присутствует среди k-ближайших к целевому, и затем причисляем его к тому классу, экземпляров которого больше всего.

```
def prediction(neighbors):

count = {}

for instance in neighbors:

if instance in count:

count[instance] +=1

else :

count[instance] = 1

target = max(count.items(), key=lambda x: x[1])[0]

return target
```

Напишем последнюю **функцию для оценки точности прогнозов**, чтобы измерить отношение верных прогнозов к общему количеству прогнозов:

```
def accuracy(test, test_prediction):

correct = 0

for i in range (len(test)):

if test[i][-1] == test_prediction[i]:

correct += 1

return (correct / len(test))
```

Посмотрим, как работает алгоритм:

```
predictions = []

for x in range (len(x_test_concat)):

neighbors = get_neighbors(x_train_concat, x_test_concat[x],
k=5)

result = prediction(neighbors)

predictions.append(result)

# print(f'predicted = {result}, actual =
{x_test_concat[x][-1]}') # Если интересно посмотреть, какие
конкретно прогнозы некорректны

accuracy = accuracy(x_test_concat, predictions)

print(f'Accuracy: {accuracy}')
```

Теперь импортируем библиотечную версию алгоритма:

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
```

Объект *knn* инкапсулирует алгоритм, который будет использоваться для построения модели из обучающих данных, а также для прогнозирования новых точек данных. Он также будет содержать информацию, которую алгоритм извлек из обучающих данных. У KNeighborsClassifier он будет просто хранить учебный набор.

Для построения модели на обучающем множестве вызывается метод fit объекта knn. Он принимает в качестве аргументов массив NumPy x_train, содержащий обучающие данные, и массив NumPy y_train соответствующих обучающих меток.

```
knn_model = knn.fit(x_train, y_train)
```

Для предсказаний вызывается метод predict, который в качестве аргументов принимает тестовые данные:

```
knn_predictions = knn.predict(x_test)

knn_predictions
```

```
array([2, 1, 0, 2, 0, 2, 0, 1, 1, 1, 2, 1, 1, 1, 1, 0, 1, 1, 0, 0, 2, 1,
       0, 0, 2, 0, 0, 1, 1, 0, 2, 1, 0, 2, 2, 1, 0, 2])
```

Для проверки импортируем простую встроенную метрику accuracy_score, которая определяет долю правильных ответов:

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, knn_predictions)

print(f'Accuracy: {accuracy}')
```

```
Accuracy: 0.9736842105263158
```

Мы получили идентичные результаты работы алгоритма, поэтому можно сделать вывод — визуализацию можно использовать для проверки работы алгоритма.

Мы видим, что реализованный вручную алгоритм при данных параметрах по точности сопоставим с библиотечной моделью. Однако на практике стоит пользоваться именно готовыми реализациями, так как зачастую они гораздо лучше оптимизированы и быстрее работают с большими выборками данных.

Интересно, что в нашем случае с помощью визуализации мы смогли точно установить, что модель сможет хорошо классифицировать имеющиеся данные. Иногда достаточно построить визуализацию, чтобы понять, что нет никакого смысла тратить время и силы на построение моделей.

Итак, мы реализовали модель, которую сможет использовать лаборатория, чтобы классифицировать те образцы ириса, которые еще не распределены. Но насколько сильно мы можем доверять нашей модели? Когда она работает лучше, а когда хуже? Возможно, стоит использовать другую модель?

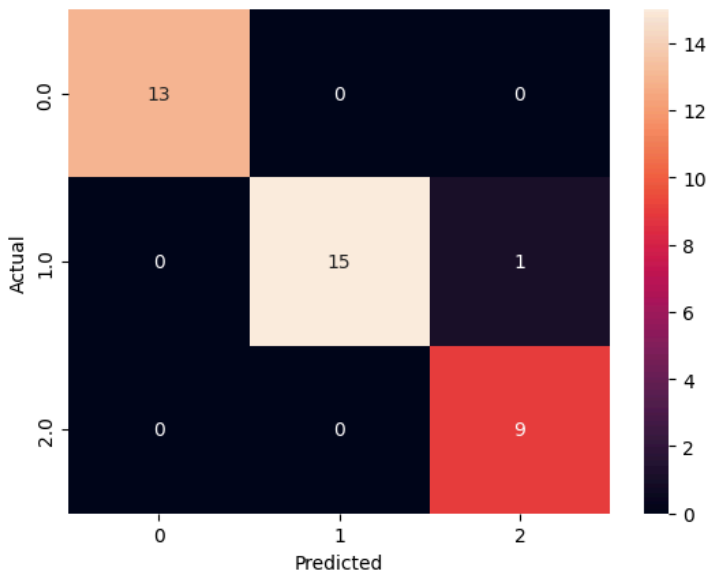
3.2. Визуализация полученных моделью результатов и Decision Tree

Нам может пригодиться более подробная статистика по качеству модели, чем показатель Accuracy. Часто нам интересно количество False Positive и False Negative.

Например, в нашем датасете 95% наблюдений относятся к первому классу (шум), а оставшиеся 5% — это те, которые нас интересуют (второй класс). После обучения модель выбрала стратегию и относит все наблюдения к первому классу. В таком случае Accuracy модели 95%, но фактически такая модель бесполезна и не помогает.

Тогда на помощь приходит построение визуализации sns.heatmap:

```
import seaborn as sns

import matplotlib.pyplot as plt

%matplotlib inline

# Создаем столбец с предсказаниями на основе наших фичей
predicted_classes = knn_model.predict(x_test_concat_df.iloc[:,
:2])

# Добавляем получившийся столбец в конец нашего датасета
x_test_concat_df['Predicted_Class'] = predicted_classes

# Создаем на основе датасета матрицу соответствия
confusion_matrix = pd.crosstab(x_test_concat_df[2],
x_test_concat_df['Predicted_Class'], rownames=['Actual'],
colnames=['Predicted'])

# Строим sns.heatmap
sns.heatmap(confusion_matrix, annot=True, fmt='d')

plt.show()
```



В нашем случае всего одно ложное срабатывание — первый класс был определен как второй. Если вспомнить проведенную ранее визуализацию, то становится понятно, почему так произошло. Эти два класса пересекаются — самые большие образцы первого класса находятся очень близко к самым маленьким образцам второго. Поэтому в одном из шестнадцати случаев наша модель все-таки ошиблась.

Мы разобрались, как использовать визуализацию, чтобы лучше использовать и понимать алгоритмы классификации.

Decision Tree и его визуализация

Попробуем решить ту же задачу, используя другие методы классификации. Один из самых популярных — **алгоритм случайного леса**. С его помощью можно получить достаточно точные результаты.

Однако есть определенная проблема — из-за внутренней структуры алгоритма, его достаточно сложно визуализировать. Однако это возможно. Попробуем это сделать:

```
# Применим алгоритм решающего дерева к нашему датасету

from sklearn.tree import DecisionTreeClassifier

from sklearn import tree


X = iris_dataset.data
```

```
y = iris_dataset.target

clf = DecisionTreeClassifier(random_state=1234)

model = clf.fit(X, y)
```

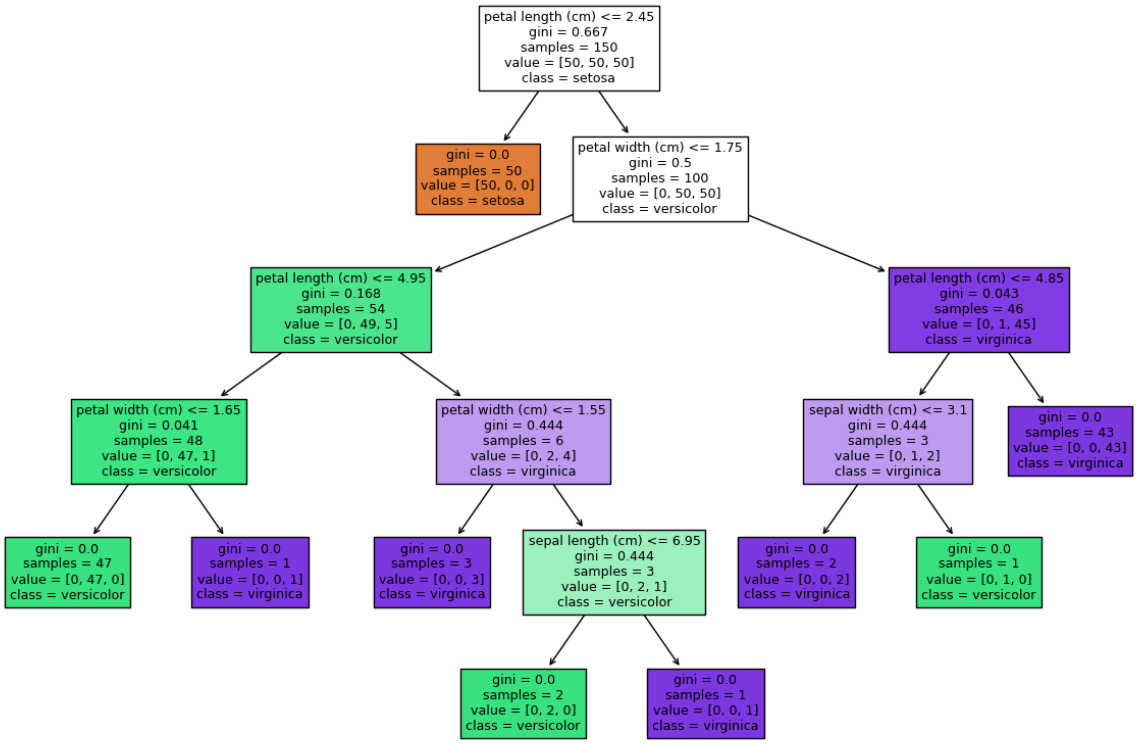
```
# Попробуем посмотреть на полученные результаты в текстовом виде

text_representation = tree.export_text(clf)

print(text_representation)
```

```
|--- feature_2 <= 2.45 | |--- class: 0 |--- feature_2 > 2.45 | |---
feature_3 <= 1.75 | | |--- feature_2 <= 4.95 | | | |--- feature_3 <=
1.65 | | | | |--- class: 1 | | | |--- feature_3 > 1.65 | | | | |---
class: 2 | | |--- feature_2 > 4.95 | | | |--- feature_3 <= 1.55 | | | |
|--- class: 2 | | | |--- feature_3 > 1.55 | | | | |--- feature_0 <=
6.95 | | | | | |--- class: 1 | | | | |--- feature_0 > 6.95 | | | | |
|--- class: 2 | |--- feature_3 > 1.75 | | |--- feature_2 <= 4.85 | | |
|--- feature_1 <= 3.10 | | | | |--- class: 2 | | | |--- feature_1 >
3.10 | | | | |--- class: 1 | | |--- feature_2 > 4.85 | | | |--- class:
2
```

Как мы видим, полученные результаты можно интерпретировать исходя из текста, но это очень трудно — ветвящаяся структура сильно усложняет восприятие. На помощь приходит визуализация — то же самое возможно изобразить с помощью диаграммы в более читаемом виде. Уже созданный пресет такой диаграммы реализован в классе tree библиотеки scikit-learn.

Создадим подобную диаграмму:

```
fig = plt.figure(figsize=(15,10))

_ = tree.plot_tree(clf,

feature_names=iris_dataset.feature_names,

class_names=iris_dataset.target_names,

filled=True)
```



Получилось намного лучше. Теперь можно отследить принцип работы алгоритма в удобном и наглядном виде.

Конечно, у этого метода есть свои ограничения. Например, если дерево слишком большое, с большим количеством признаков, то визуализация получится слишком громоздкой и сложно интерпретируемой.

Полученные результаты можно сохранить как png-картинку с помощью кода:

```
fig.savefig("decistion_tree.png")
```

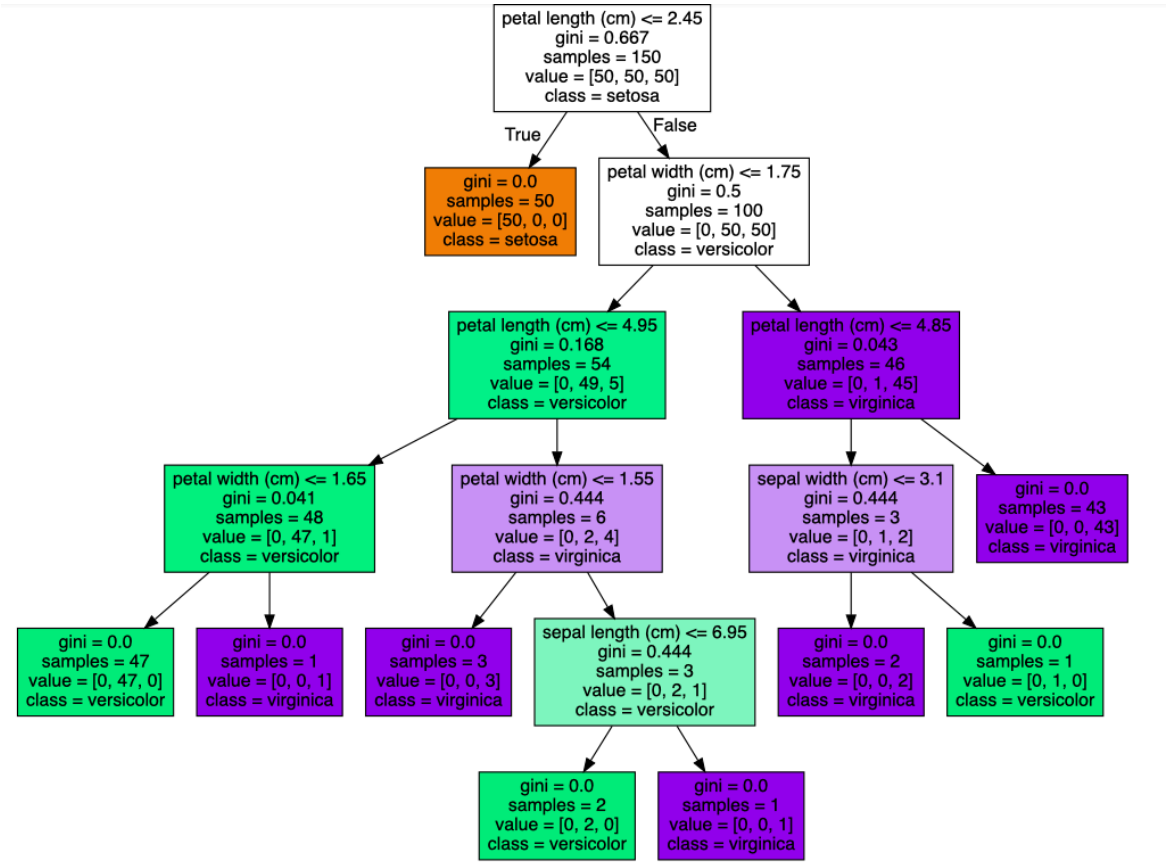
Есть и другие библиотеки, которые позволят визуализировать полученные результаты. Можете выбрать любую, которая вам больше нравится.

Первая библиотека называется **graphviz**:

```
# Если не установлен данный модуль

!pip install graphviz
```

```
account_circle

Requirement already satisfied: graphviz in
/usr/local/lib/python3.10/dist-packages (0.20.1)
```

```
import graphviz

dot_data = tree.export_graphviz(clf, out_file=None,
feature_names=iris_dataset.feature_names,
class_names=iris_dataset.target_names,
filled=True)

# Draw graph
graph = graphviz.Source(dot_data, format="png")
graph
```



Полученные результаты хоть и похожи на те, что предоставляет scikit-learn, но более сжаты и лаконичны. Кроме того, здесь эксплицитно указано, какая стрелка отвечает за выполнение условия True, а какая — за False.

Есть еще один популярный модуль, который сильно отличается от двух предыдущих — **dtreeviz**. Он представляет собой наиболее продвинутые методы визуализации решающих деревьев и некоторых других методов.

К сожалению, существуют сложности с визуализацией алгоритма случайного леса (совокупности разных деревьев). Однако всегда можно визуализировать каждое решающее дерево по отдельности.

Некоторые другие методы также поддаются классификации, например, Support Vector Machines (SVM). Но мы не будем рассматривать их отдельно.

3.3. Визуализация результатов работы алгоритма градиентного спуска

Один из важных алгоритмов машинного обучения — **градиентный спуск**. Основная его проблема — он предназначен для нахождения локального минимума или максимума, а не глобального. Однако алгоритм очень полезен в ряде задач.

Самостоятельно напишем алгоритм градиентного спуска и создадим динамическую визуализацию, которая позволит следить за работой алгоритма. Для начала реализуем функции, необходимые нам для визуализации.

Визуализация функции одной переменной:

```
def plot_1d_fuction(xs: list, ys: list, title: str,
figsize=(15, 5)):
'''
xs — значения функции по x
ys — значения функции по y
title — заголовок картинки
msg — информация для отображения на картинке
figsize — размер картинки
'''

plt.figure(figsize=figsize)
plt.plot(xs, ys, 'b')
plt.title(title)
plt.ylabel('$f(x)$')
plt.xlabel('$x$');
```

Функция для отрисовки графика и значение функции в точке x:

```
from IPython.display import clear_output

def plot_line_with_x(xs: list, ys: list, x: float, fx: float,
title: str, msg='', figsize=(15, 5)):
'''
xs — значения функции по x
ys — значения функции по y
x — текущее положение точки по x
fx — текущее положение точки по y
title — заголовок картинки
msg — информация для отображения на картинке
figsize — размер картинки
'''

clear_output(True)
plt.figure(figsize=figsize)
plt.plot(xs, ys, 'b')
plt.scatter([x], [fx], c='r', marker='x', s=120)

plt.title(title)
plt.ylabel('$f(x)$')
plt.xlabel('$x$')
plt.text(s=msg, x=-2, y=103)
plt.show()
```

Начнем с поиска минимума полинома шестой степени:

$$(f(x) = x^6 + 3x^5 - 15x^3 - 24x^2 + 15x - 10)$$

Для этого воспользуемся стандартным алгоритмом градиентного спуска, для которого понадобится найти производную.

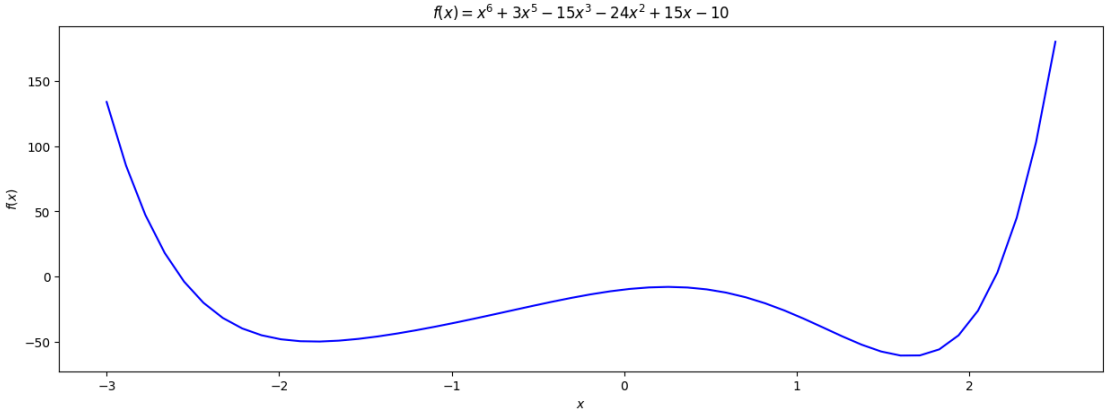
Реализуем функцию вычисления значений $f(x)$:

```
def f_1d(x):

return (x ** 6) + 3 * (x ** 5) - 15 * (x ** 3) - 24 * (x ** 2)
+ 15 * x - 10
```

Нарисуем график функции на отрезке от -3 до 2.5:

```
title_f_1d = '$f(x) = x^{6} + 3x^{5} - 15x^{3} - 24x^{2} + 15x
- 10$'
```

```
xs = np.linspace(-3, 2.5)
ys = f_1d(xs)

plot_1d_fuction(xs, ys, title=title_f_1d)
```



Мы видим, что у функции два минимума.

Алгоритм градиентного спуска в самом общем виде выглядит так:

```
until converge:

    x = x - alpha * grad(f(x))
```

Где alpha — это скорость обучения (то, как сильно мы меняем свое положение на каждом шаге), а `grad(f(x)) — производная от функции $f(x)$.

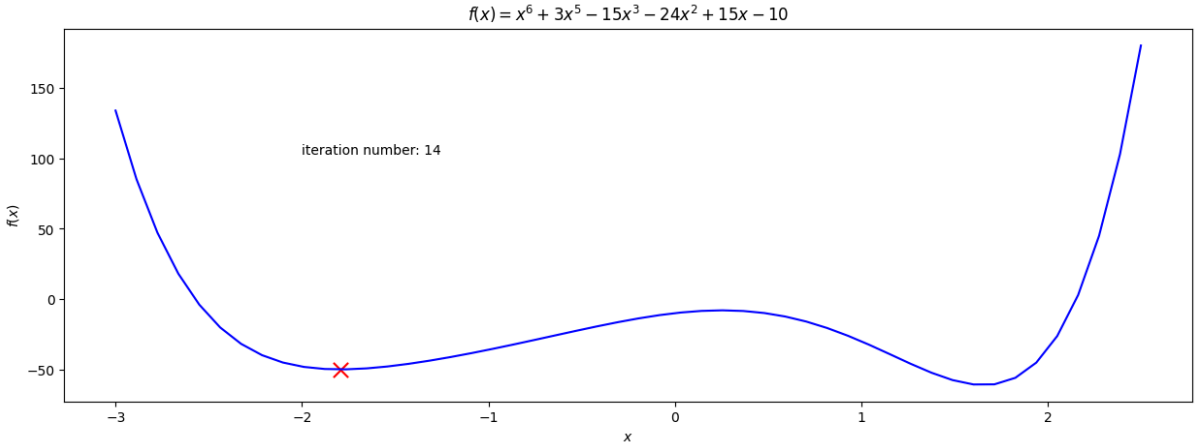
Реализуем функцию для вычисления производной в нужной нам точке. Для этого используем школьный алгоритм производной степени:

```
def grad(x):

return 6 * x ** 5 + 15 * x ** 4 - 45 * x ** 2 - 48 * x + 15
```

Все готово для поиска минимума функции методом градиентного спуска.

Начнем с точки $x_0 = 0.2$, величину шага α примем равной 0.03 и будем останавливать итерации, если абсолютное значение градиента не превышает 10^{-4} (параметр tol).

```
alpha = 0.03
tol = 1e-4

x = -3

max_iteration = 200
iteration = 0

dfdx = np.clip(grad(x), -10, 10)
dfdx_sign = np.sign(dfdx)

for i in range(max_iteration):
if abs(dfdx) < tol:
break

dfdx = np.clip(grad(x), -10, 10)
x = x - alpha * dfdx
iteration += 1

if dfdx_sign != np.sign(dfdx):
alpha = alpha / 1.3

dfdx_sigh = np.sign(dfdx)
plot_line_with_x(xs, ys, x, f_1d(x), msg=f'iteration number: {iteration}',
title=title_f_1d)
```



Можно поиграть с параметром x и посмотреть результаты, которые получаются в графике, если изменить начальную точку.

Выводы

- Для анализа полученных алгоритмом классификации результатов обычно используют так называемую Confusion Matrix (Матрица Несоответствий, создается с помощью pd.crosstab). Эта таблица нужна, чтобы отслеживать классы, которые были неправильно атрибутированы и где возникает главная путаница. Чтобы лучше проанализировать и визуализировать Confusion Matrix, используют sns.heatmap().
- Decision Tree — один из самых сложных для визуализации алгоритмов. Это связано с архитектурой алгоритма. Получить схему с результатами можно с помощью функции tree.export_text(df).
- Но такой способ неудобный — в такой схеме очень сложно разобраться в результатах. На помощь приходит функция tree.plot_tree() библиотеки sklearn, позволяющая визуализировать полученные результаты в виде нисходящего дерева вариантов. Более продвинутую и удобную версию такой визуализации предоставляет библиотека graphviz, благодаря которой можно получить подробные сведения о результатах работы модели.
- С помощью линейного графика можно создать визуализацию работы градиентного спуска.

Дополнительная литература

- pandas.plot argument c vs s
- GitHub'е авторов dtreeviz
- SVM 2Dimension-Hyperplane: Visualization
- Градиентный спуск простыми словами
- Метод pandas.plotting
- Визуализация данных: когда использовать тот или иной способ оценки качества классификации